第三节 四步构建RAG

通过第一节的学习，我们对RAG已经有了基本认识，并且也准备好了虚拟环境和api_key，接下来将尝试使用[LangChain]和[LlamaIndex]框架完成第一个RAG应用的实现与运行。通过一个示例，演示如何加载本地Markdown文档，利用嵌入模型处理文本，并结合大型语言模型（LLM）来回答与文档内容相关的问题。

一、启动虚拟环境

1.1 激活虚拟环境

假设已经按照前一章节的指导，创建了名为 `all-in-rag` 的 Conda 虚拟环境。在运行脚本前，先激活虚拟环境：

> 如果使用是Cloud Studio，需要确认当前是否是用户环境，如果不是请运行 `su ubuntu` 切换到用户环境。

```
conda activate all-in-rag
```

1.2 切换到项目目录

```
# 假设当前在 all-in-rag 项目的根目录下
cd code/C1
```

每章内容中的代码文件都存放在 `code/Cx` 目录下，其中 `x` 表示章节编号。

二、运行RAG示例代码

完成上述所有设置后，就可以运行RAG示例了。

打开终端，确保虚拟环境已激活，然后执行以下命令：

```
python 01_langchain_example.py
```

> 若出现nltk相关报错，尝试运行代码路径下fix_nltk.py

代码运行后，可以看到类似下面的输出（格式化后）：

```
Downloading Model from https://www.modelscope.cn to directory: Path\to\all-in-rag\models\bge-small-zh-v1.5
2025-06-08 02:36:19,318 - modelscope - INFO - Target directory already exists, skipping creation.
content='
文中举了以下例子：

1. **自然界中的羚羊**：刚出生的羚羊通过试错学习站立和奔跑，适应环境。
2. **股票交易**：通过买卖股票并根据市场反馈调整策略，最大化奖励。
3. **雅达利游戏（如Breakout和Pong）**：通过不断试错学习如何通关或赢得游戏。
4. **选择餐馆**：利用（去已知喜欢的餐馆）与探索（尝试新餐馆）的权衡。
5. **做广告**：利用（采取已知最优广告策略）与探索（尝试新广告策略）。
```

```
6. **挖油**：利用（在已知地点挖油）与探索（在新地点挖油，可能发现大油田）。
7. **玩游戏（如《街头霸王》）**：利用（固定策略如蹲角落出脚）与探索（尝试新招式如“大招”）。

这些例子用于说明强化学习中的核心概念（如探索与利用、延迟奖励等）及其在实际场景中的应用。
'
additional_kwargs={'refusal': None}
response_metadata={
    'token_usage': {
        'completion_tokens': 209,
        'prompt_tokens': 5576,
        'total_tokens': 5785,
        'completion_tokens_details': None,
        'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 5568},
        'prompt_cache_hit_tokens': 5568,
        'prompt_cache_miss_tokens': 8
    },
    'model_name': 'deepseek-chat',
    'system_fingerprint': 'fp_8802369eaa_prod0425fp8',
    'id': '67a0580d-78b1-44d6-bccf-f654ae0e9bba',
    'service_tier': None,
    'finish_reason': 'stop',
    'logprobs': None
}
id='run--919cedcd-771e-4aed-8dfd-cf436795792e-0'
usage_metadata={
    'input_tokens': 5576,
    'output_tokens': 209,
    'total_tokens': 5785,
    'input_token_details': {'cache_read': 5568},
    'output_token_details': {}
}
```

> 首次运行时，脚本会下载`BAAI/bge-small-zh-v1.5`嵌入模型。

输出参数解析：

- `content`：这是最核心的部分，即大型语言模型（LLM）根据你的问题和提供的上下文生成的具体回答。
- `additional_kwargs`：包含一些额外的参数，在这个例子中是`{'refusal': None}`，表示模型没有拒绝回答。
- `response_metadata`：包含了关于LLM响应的元数据。
  - `token_usage`：显示了本次调用消耗的token数量，包括完成（completion_tokens）、提示（prompt_tokens）和总量（total_tokens）。
  - `model_name`：使用的LLM模型名称，当前是`deepseek-chat`。
  - `system_fingerprint`，`id`，`service_tier`，`finish_reason`，`logprobs`：这些是更详细的API响应信息，例如`finish_reason: 'stop'`表示模型正常完成了生成。
- `id`：本次运行的唯一标识符。
- `usage_metadata`：与`response_metadata`中的`token_usage`类似，提供了输入和输出token的统计。

三、基于LangChain框架的RAG实现

> 在第一节中，我们提到四步构建最小可行系统分别是数据准备、索引构建、检索优化和生成集成。接下来将围绕这四个方面来实现一个基于LangChain框架的RAG应用。

3.1 初始化设置

首先进行基础配置，包括导入必要的库、加载环境变量以及下载嵌入模型。

```
import os
# os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
from dotenv import load_dotenv
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_core.prompts import ChatPromptTemplate
from langchain_deepseek import ChatDeepSeek

# 加载环境变量
load_dotenv()
```

3.2 数据准备 (Data Preparation)

- **加载原始文档**: 先定义Markdown文件的路径，然后使用`TextLoader`加载该文件作为知识源。

  ```
  markdown_path = "../../data/C1/markdown/easy-rl-chapter1.md"
  loader = TextLoader(markdown_path)
  docs = loader.load()
  ```

- **文本分块 (Chunking)**: 为了便于后续的嵌入和检索，长文档被分割成较小的、可管理的文本块（chunks）。这里采用了递归字符分割策略，使用其默认参数进行分块。当不指定参数初始化`RecursiveCharacterTextSplitter()`时，其默认行为旨在最大程度保留文本的语义结构：
  - **默认分隔符与语义保留**: 按顺序尝试使用一系列预设的分隔符`["\n\n" (段落), "\n" (行), " " (空格), "" (字符)]`来递归分割文本。这种策略的目的是尽可能保持段落、句子和单词的完整性，因为它们通常是语义上最相关的文本单元，直到文本块达到目标大小。
  - **保留分隔符**: 默认情况下(`keep_separator=True`)，分隔符本身会被保留在分割后的文本块中。
  - **默认块大小与重叠**: 使用其基类`TextSplitter`中定义的默认参数`chunk_size=4000`（块大小）和`chunk_overlap=200`（块重叠）。这些参数确保文本块符合预定的大小限制，并通过重叠来减少上下文信息的丢失。

  ```
  text_splitter = RecursiveCharacterTextSplitter()
  texts = text_splitter.split_documents(docs)
  ```

3.3 索引构建 (Index Construction)

数据准备完成后，接下来构建向量索引：

- **初始化中文嵌入模型**: 使用`HuggingFaceEmbeddings`加载之前在初始化设置中下载的中文嵌入模型。配置模型在CPU上运行，并启用嵌入归一化(`normalize_embeddings: True`)。

```
embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-small-zh-v1.5",
    model_kwargs={'device': 'cpu'},
    encode_kwargs={'normalize_embeddings': True}
)
```

- **构建向量存储**: 将分割后的文本块 (`texts`) 通过初始化好的嵌入模型转换为向量表示，然后使用 `InMemoryVectorStore` 将这些向量及其对应的原始文本内容添加进去，从而在内存中构建出一个向量索引。

  ```
  vectorstore = InMemoryVectorStore(embeddings)
  vectorstore.add_documents(texts)
  ```

  这个过程完成后，便构建了一个可供查询的知识索引。

3.4 查询与检索 (Query and Retrieval)

索引构建完毕后，便可以针对用户问题进行查询与检索：

- **定义用户查询**: 设置一个具体的用户问题字符串。

  ```
  question = "文中举了哪些例子？"
  ```

- **在向量存储中查询相关文档**: 使用向量存储的 `similarity_search` 方法，根据用户问题在索引中查找最相关的 `k` (此处示例中 `k=3`) 个文本块。

  ```
  retrieved_docs = vectorstore.similarity_search(question, k=3)
  ```

- **准备上下文**: 将检索到的多个文本块的页面内容 (`doc.page_content`) 合并成一个单一的字符串，并使用双换行符 (`"\n\n"`) 分隔各个块，形成最终的上下文信息 (`docs_content`) 供大语言模型参考。

  ```
  docs_content = "\n\n".join(doc.page_content for doc in retrieved_docs)
  ```

  > 使用 `"\n\n"` (双换行符) 而不是 `"\n"` (单换行符) 来连接不同的检索文档块，主要是为了在传递给大型语言模型（LLM）时，能够更清晰地在语义上区分这些独立的文本片段。双换行符通常代表段落的结束和新段落的开始，这种格式有助于LLM将每个块视为一个独立的上下文来源，从而更好地理解和利用这些信息来生成回答。

3.5 生成集成 (Generation Integration)

最后一步是将检索到的上下文与用户问题结合，利用大语言模型（LLM）生成答案：

- **构建提示词模板**: 使用 `ChatPromptTemplate.from_template` 创建一个结构化的提示模板。此模板指导LLM根据提供的上下文 (`context`) 回答用户的问题 (`question`)，并明确指出在信息不足时应如何回应。

```
prompt = ChatPromptTemplate.from_template("""请根据下面提供的上下文信息来回答问题。
请确保你的回答完全基于这些上下文。
如果上下文中没有足够的信息来回答问题，请直接告知：“抱歉，我无法根据提供的上下文找到相关信息来回答此问题。”

上下文:
{context}

问题: {question}

回答:"""
                                                    )
```

- **配置大语言模型**: 初始化`ChatDeepSeek`客户端，配置所用模型(`deepseek-chat`)、生成答案的温度参数(`temperature=0.7`)、最大Token数(`max_tokens=2048`)以及API密钥(从环境变量加载)。

```
llm = ChatDeepSeek(
    model="deepseek-chat",
    temperature=0.7,
    max_tokens=2048,
    api_key=os.getenv("DEEPSEEK_API_KEY")
)
```

- **调用LLM生成答案并输出**: 将用户问题(`question`)和先前准备好的上下文(`docs_content`)格式化到提示模板中，然后调用ChatDeepSeek的`invoke`方法获取生成的答案。

```
answer = llm.invoke(prompt.format(question=question, context=docs_content))
print(answer)
```

完整代码

> 老湿老湿，Langchain很强大但还是太吃操作了，有没有更加简单又好用的框架推荐呢?

> 有的兄弟，有的！像这样好用的框架还有LlamaIndex😉

四、低代码（基于LlamaIndex）

在RAG方面，LlamaIndex提供了更多封装好的API接口，这无疑降低了上手门槛，下面是一个简单实现:

```
import os
# os.environ['HF_ENDPOINT']='https://hf-mirror.com'
from dotenv import load_dotenv
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.deepseek import DeepSeek
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

load_dotenv()

Settings.llm = DeepSeek(model="deepseek-chat",
api_key=os.getenv("DEEPSEEK_API_KEY"))
Settings.embed_model = HuggingFaceEmbedding("BAAI/bge-small-zh-v1.5")
```

```
documents = SimpleDirectoryReader(input_files=["../../data/C1/markdown/easy-rl-chapter1.md"]).load_data()

index = VectorStoreIndex.from_documents(documents)

query_engine = index.as_query_engine()

print(query_engine.get_prompts())

print(query_engine.query("文中举了哪些例子?"))
```

练习（可利用大模型辅助完成）

- LangChain代码最终得到的输出携带了各种参数，查询相关资料尝试把这些参数过滤掉得到`content`里的具体回答。
- 修改Langchain代码中`RecursiveCharacterTextSplitter()`的参数`chunk_size`和`chunk_overlap`，观察输出结果有什么变化。
- 给LlamaIndex代码添加代码注释。